

# # Agriculture Tracker — Project Report

Date: 2026-06-02 Author: Generated by assistant

## 1. High-level Overview

Agriculture Tracker is a Laravel 12 application for managing inventory, purchases, sales, suppliers, customers, and reports. It includes role/permission based access control (Spatie), export features (CSV/XLSX), and a Progressive Web App (PWA) front-end built with Vite and vite-plugin-pwa to enable "installable" and offline-capable behavior on mobile devices.

Primary features: - User authentication and profile management - Role & permission management (spatie/laravel-permission) - CRUD for categories, products, suppliers, customers, purchases, sales - Sales invoicing and purchase invoicing (PDF/HTML views) - Credit payments tracking and customer due reports - Reports: aggregated sales summaries, monthly sales, low-stock alerts - Export: CSV streams and Excel workbook export - PWA: manifest, service worker, offline fallback, caching strategies

## 2. Architecture & Tech Stack

- Backend: Laravel 12 (PHP 8.2+) - Frontend: Blade templates + Vite (Tailwind + Alpine.js) - PWA: vite-plugin-pwa (service worker generated at build) - Auth scaffold: Laravel Breeze / built-in auth - Permissions: spatie/laravel-permission - Excel export: maatwebsite/excel - Exports/Reports: CSV and Excel, streamed responses - DB: Eloquent models with migrations (MySQL / SQLite possible)

Directory highlights: - app/Models/ — Eloquent models (Product, Sale, Purchase, Customer, Supplier, Category...) - app/Http/Controllers/ — Controllers implementing business logic and exports - resources/views/ — Blade views and layouts - routes/web.php — Application routes and middleware (permission checks) - public/ — Built assets, PWA files (manifest.webmanifest, sw.js, offline.html)

## 3. Data Model Summary

Key models and relations (simplified):

- Product - belongsTo Category - hasMany PurchaseItem, SaleItem - attributes: stock\_quantity, purchase\_price, selling\_price, low\_stock\_threshold

- Purchase - belongsTo Supplier, User - hasMany PurchaseItem - attributes: invoice\_no, purchase\_date, total\_amount

- Sale - belongsTo Customer, User - hasMany SaleItem, CreditPayment - attributes: invoice\_no, sale\_date, payment\_type, subtotal, discount, total\_amount, paid\_amount, due\_amount, payment\_status - method: updatePaymentStatus() sets payment\_status to paid/partial/unpaid based on paid/due amounts

- Customer, Supplier, Category — standard CRUD fields (name, contact, address) - SaleItem / PurchaseItem store per-line product, qty, unit price, totals - CreditPayment stores payments made against credit sales

## 4. Routes & Permissions

All protected routes use middleware auth and verified. Additionally, Spatie permission middleware restricts access — examples from routes/web.php:

- dashboard route requires permission:dashboard.view - resource('products') requires permission:products.manage - resource('purchases') requires permission:purchases.manage - resource('sales') requires permission:sales.manage - reports require permission:reports.view - users require permission:users.manage

Permissions are managed via seeder database/seeders/RolePermissionSeeder.php and config in config/permission.php.

## 5. Important Controllers & Business Logic

- ReportController — composes queries for sales within a date range, computes summary metrics (total sales, cash vs credit, totals paid/pending), builds exports: - exportSalesCsv() — streams CSV of sales - exportCreditCustomersCsv() — streams CSV of credit customers with pending dues - exportExcel() — uses ReportsWorkbookExport for XLSX
- PurchaseController / SaleController — handle creation/updating of transactions, computing totals, storing line items, generating invoice views
- ProductController — manages inventory and stock quantities; low stock detection used in reports.
- CreditPaymentController — receives payments for credit sales and updates corresponding sale paid\_amount and due\_amount and status.

## 6. PWA Implementation (what was added)

- vite-plugin-pwa installed and configured in vite.config.js. - Generated manifest.webmanifest and sw.js under public/build after npm run build. - resources/views/partials/pwa-head.blade.php included in app layouts to reference manifest and icons. - Service worker registered in resources/js/app.js via virtual:pwa-register. - Offline fallback: public/offline.html and Workbox navigateFallback configured, plus offline.html precached. - Caching strategies: NetworkFirst for navigation (pages) and StaleWhileRevalidate for static assets.

Notes for viva: explain that PWA allows app-like behavior, install prompt, offline access to app shell and previously visited pages.

## 7. How to run locally (development)

Requirements: PHP 8.2+, Composer, Node.js, npm.

### 1. Clone repo

### 2. For PWA testing locally (quick): create a public tunnel (we used localtunnel or ngrok):

Open the provided https://... URL on mobile and use "Add to Home screen" / "Install app".

## 8. How to run in production (concise)

- Build assets: npm run build - Upload project to server (SSH/FTP/Git) - Configure .env with production DB and APP\_URL=https://your-domain.com - Composer install and Laravel migrations: composer install --no-dev, php artisan migrate --force, php artisan storage:link - Serve with Nginx + PHP-FPM; enable HTTPS (Let's Encrypt / certbot)

I can provide a full server deployment script (Ubuntu Nginx) on request.

## 9. Tests, Exports, Reports — Viva pointers

- Functionality tests (manual): create products, create purchases (increase stock), create sales (decrease stock), create credit sale (payment\_type=credit) and then add CreditPayment to change due\_amount. - Exports: use Reports page to export CSV / Excel. ReportController uses streaming for CSV to avoid memory blowup. - Low-stock: Product::whereColumn('stock\_quantity', '<=', 'low\_stock\_threshold') used to detect low inventory.

## 10. Known limitations & next steps

- Offline: only app shell and cached pages available; full offline CRUD with background sync is not implemented. To support offline writes, implement a client-side queue (IndexedDB) and background sync to send queued operations to server when online. - Mobile iOS: PWA features on iOS Safari are limited (no background sync, limited storage); highlight in viva. - Security: ensure APP\_DEBUG=false in production and follow common Laravel hardening.

## 11. Appendix — Useful Files & Commands

- routes/web.php — main routes and permission gating - vite.config.js — PWA plugin configuration - resources/js/app.js — service worker registration - public/offline.html — offline fallback page - Build commands:

Convert this report to PDF (locally)

Install pandoc and a PDF engine (wkhtmltopdf or TeX Live). Example using pandoc + wkhtmltopdf:

Or use pandoc with LaTeX:

--- If you want, I can: - Generate a printable PDF in the workspace (I can produce Markdown here; you'll need to run pandoc locally), or - Walk you through deploying to Oracle Cloud step-by-step and make URL public (I can run remote commands if you give SSH key and permit), or - Prepare a short viva script (Q&A style) summarizing likely exam questions and sample answers.

Tell me which of the above you want next.